
Advanced Compiler Optimizations in Soot

Redundant Load Detection via Points-To Analysis (PA2)

Utkarsh Shetye

CS6004: Code Optimization for Object-Oriented Languages

Abstract

Object-oriented languages heavily rely on heap allocations, polymorphic dispatch, and frequent object field accesses. While these features enable powerful abstractions, they introduce significant runtime overhead through redundant memory loads, cache thrashing, and garbage collection pauses. This report details the implementation of an advanced static analysis pass developed using the Soot compiler framework.

This phase (PA2) targets **Redundant Load Detection**. By constructing a flow-sensitive and field-sensitive intraprocedural points-to graph, the analysis identifies field access operations that retrieve unchanged data already available in local variables, pinpointing exact source-code locations for optimization.

Contents

1	Introduction and Motivation	3
1.1	The Cost of Object-Oriented Memory Paradigms	3
1.2	Project Scope	3
2	Theoretical Background: Points-To Analysis	3
2.1	Flow-Sensitive Intraprocedural Points-To Analysis	3
2.2	Strong vs. Weak Updates	3
3	Redundant Load Detection Implementation	5
3.1	Objectives and Constraints	5
3.2	Architecture of the AnalysisTransformer	5
3.2.1	The <code>PointToGraph</code> Data Structure	5
3.2.2	Flow Equations and Transfer Functions	5
3.2.3	Conservative Invalidation on Method Calls	6
3.3	Output Generation	6
4	Design Challenges and Optimizations	7
4.1	Alias Invalidation Completeness	7
5	Conclusion	7

1 Introduction and Motivation

1.1 The Cost of Object-Oriented Memory Paradigms

In Java, virtually all objects are allocated on the managed heap. While the JVM provides a highly optimized Garbage Collector (GC), heap allocation is inherently more expensive than stack allocation. Every object creation involves securing contiguous memory, updating metadata headers, and eventually paying the cost of GC marking and sweeping when the object becomes unreachable.

Furthermore, within tightly bound loops or complex control-flow branches, programs frequently access the same object fields repeatedly. If the field is not mutated between accesses, these redundant load operations issue unnecessary instructions to the memory hierarchy. Even if the data is served from the L1 CPU cache, the sheer volume of redundant instructions degrades instruction-level parallelism and pollutes the Intermediate Representation (IR) with noise that hinders further downstream optimizations.

1.2 Project Scope

To combat these inefficiencies, this project implements a distinct data-flow analysis using the **Soot Framework**:

PA2: Let Go of Them (Redundant Load Elimination) - An intraprocedural analysis that computes available loads over a points-to graph to flag repetitive $\mathbf{x} = \mathbf{o.f}$ statements.

2 Theoretical Background: Points-To Analysis

2.1 Flow-Sensitive Intraprocedural Points-To Analysis

A points-to analysis determines the set of abstract memory locations (objects) that a pointer variable may refer to during execution. In this project, the analysis is *flow-sensitive*, meaning it respects the control-flow order of statements, maintaining distinct states for every node in the Control Flow Graph (CFG).

The analysis is formalized as a lattice of maps:

$$PTG = (V \rightarrow \mathcal{P}(O)) \times (O \times F \rightarrow \mathcal{P}(O)) \quad (1)$$

Where V is the set of local variables, O is the set of allocation sites (abstract objects), and F is the set of field signatures.

2.2 Strong vs. Weak Updates

When an assignment of the form $\mathbf{v.f} = \mathbf{w}$ is encountered, the analysis must update the abstract field mapping.

- **Strong Update:** If v points to exactly *one* concrete object, and we are certain it refers to the most recent instance of that object, we can safely overwrite the old field mapping. This is crucial for precision.

- **Weak Update:** If v may point to multiple objects (e.g., inside a loop or after a control-flow merge), we must conservatively merge the new target w with the existing targets, performing a set union.

3 Redundant Load Detection Implementation

3.1 Objectives and Constraints

The primary goal of PA2 is to identify statements where an object field (`o.f`) is loaded into a variable (`x`), but the exact same value was previously loaded into another variable (`y`), and no intervening writes to `o.f` have occurred on any possible execution path.

3.2 Architecture of the AnalysisTransformer

The solution is built as a `BodyTransformer` in Soot, executing independently on every method body. We extend Soot's `ForwardFlowAnalysis<Unit, PointToGraph>`.

3.2.1 The PointToGraph Data Structure

To maintain the required state at each program point, the graph encapsulates:

- `objMap`: A `Map<Local, Set<Integer>>` linking local references to line numbers where allocations occurred.
- `fieldMap`: A `Map<String, Set<Integer>>` tracking what objects are held within specific fields.
- `loads`: A `Map<String, Local>` acting as the "Available Loads" cache. It maps an abstract field signature (e.g., `o10.f`) to the local variable currently holding its value.
- `aliasMap`: A `Map<Local, Local>` tracking direct reference copies (`v = w`).

3.2.2 Flow Equations and Transfer Functions

The `flowThrough()` method governs state transitions. For every unit (statement) processed, the out-state is initialized as a copy of the in-state.

Case 1: Object Allocation (`x = new T()`) The variable `x` is mapped to a new abstract object representing the current line number. Any existing alias mappings for `x` are invalidated.

Case 2: Field Read (`x = y.f`) The analysis checks the `loads` map. If `y.f` is already present, `x` becomes an alias to the cached variable. If not, `x` is recorded in the `loads` map as the new cached holder of `y.f`.

Case 3: Field Write (`x.f = y`) This requires invalidation. The analysis retrieves the points-to set of `x`.

```
1 Set<Integer> basePointer = in.objMap.getOrDefault(base, new HashSet<>());
2 Set<Integer> valuePointer = in.objMap.getOrDefault(rigLocal, new
   HashSet<>());
3
4 boolean isStrong = (basePointer.size() == 1 &&
5                     basePointer.iterator().next() > 0 &&
6                     !in.passVar.contains(basePointer.iterator().next()))
7
8 for(Integer objNo: basePointer) {
```

```

9      String key = objNo + "." + inf.getField().getSignature();
10     if (isStrong) {
11         out.fieldMap.put(key, new HashSet<>(valuePointer));
12     } else {
13         out.fieldMap.computeIfAbsent(key, k -> new HashSet<>()).addAll(
valuePointer);
14     }
15 }

```

Listing 1: Strong vs Weak Update Implementation

3.2.3 Conservative Invalidation on Method Calls

If the statement contains an `InvokeExpr`, the analysis must assume the invoked method could mutate the state of any object passed as an argument. The variables passed to the method are added to a `passVar` set. Any entry in the `loads` cache related to these escaped variables is forcefully removed to prevent unsound redundancy detection.

3.3 Output Generation

A post-processing pass (`printRedundant()`) iterates over the method statements. If an assignment `x = y.f` is encountered, and the `PointToGraph` immediately prior to this statement contains an entry in `loads` for `y.f` pointing to a variable `z`, the system flags the statement: Line N: `y.f` (can be replaced with `z`).

4 Design Challenges and Optimizations

4.1 Alias Invalidation Completeness

In PA2, ensuring that `x = y.f` is flagged redundantly requires pristine tracking of array writes and field writes. A dedicated `invalidateFields()` function was written to iterate over the `loads` map. If `base.f` is updated, not only must `base.f` be purged from the cache, but any alias of `base` must also have its corresponding `.f` entries purged. Failure to do so would result in unsound optimization recommendations.

5 Conclusion

The integration of this static analysis pass provides a powerful mechanism for automated code optimization.

Redundant Load Detection proves that intraprocedural data-flow analysis can reliably strip away redundant memory accesses caused by verbose object-oriented patterns. By carefully managing strong and weak updates, the analysis preserves soundness while maximizing precision, demonstrating a profound understanding of modern compiler architecture, intermediate representation traversal, and the intricate balancing act between analysis precision and execution soundness.